



ПРОГРАММИРОВАНИЕ НА PYTHON

Модуль 3: Профессиональный уровень

Алгоритмы • Генераторы • Декораторы • API • Базы данных

Глава 15: Генераторы и итераторы

15.1 Что такое итератор?

Итератор — объект, который умеет выдавать элементы по одному. Списки, строки, словари — всё это итерируемые объекты. Функция `iter()` и `next()` дают доступ к механизму изнутри.

```
numbers = [10, 20, 30]
it = iter(numbers)          # создаём итератор

print(next(it))             # 10
print(next(it))             # 20
print(next(it))             # 30
# print(next(it))          # StopIteration — элементы закончились
```

```
10
20
30
```

15.2 Генераторная функция (yield)

Функция с `yield` — это генератор. Она не возвращает сразу все значения, а выдаёт их по одному — это экономит память.

```
def count_up(start, end):
    current = start
    while current <= end:
        yield current        # «отдать» значение и заморозиться
        current += 1

for n in count_up(1, 5):
    print(n)

# Можно сгенерировать миллион чисел — памяти почти не нужно!
big = count_up(1, 1_000_000)
print(next(big))            # 1 — только одно число в памяти
```

```
1
2
3
4
5
1
```

❑ **Зачем нужны генераторы?** Если нужно обработать огромный файл или миллионы строк — генератор читает их по одной, не загружая всё в память сразу.

15.3 Генераторное выражение

Краткая запись генератора в одну строку — как list comprehension, но в круглых скобках:

```
# List comprehension — всё в памяти
squares_list = [x**2 for x in range(10)]

# Генераторное выражение — по одному
squares_gen = (x**2 for x in range(10))

print(sum(squares_gen))    # 285 — эффективнее!

# List comprehension с фильтром
evens = [x for x in range(20) if x % 2 == 0]
print(evens)    # [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]
```

 **Задание 1:** Напиши генератор `fibonacci(n)`, который выдаёт первые `n` чисел Фибоначчи. Выведи первые 10 через цикл `for`.

Глава 16: Декораторы и замыкания

16.1 Функции — объекты первого класса

В Python функции можно передавать как аргументы и возвращать из других функций. На этом основаны декораторы.

```
def greet():
    return "Привет!"

def shout(func):          # принимаем функцию
    result = func()
    return result.upper() # меняем результат

print(shout(greet))      # ПРИВЕТ!
```

16.2 Замыкание (closure)

Замыкание — функция, которая «помнит» переменные из внешней области видимости.

```
def make_multiplier(factor):
    def multiply(x):
        return x * factor    # помним factor!
    return multiply

double = make_multiplier(2)
triple = make_multiplier(3)

print(double(5))    # 10
print(triple(5))    # 15
```

16.3 Декоратор

Декоратор — функция, которая оборачивает другую функцию, добавляя ей новое поведение. Синтаксис @имя_декоратора — это сокращение.

```
import time

def timer(func):
    """Декоратор: измеряет время выполнения функции"""
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f"{func.__name__} выполнялась за {end-start:.4f} сек")
        return result
    return wrapper

@timer
def slow_sum(n):
    return sum(range(n))

result = slow_sum(1_000_000)
print(f"Сумма: {result}")
```

```
slow_sum выполнялась за 0.0412 сек  
Сумма: 499999500000
```

```
def require_positive(func):  
    """Декоратор: проверяет, что аргумент положительный"""  
    def wrapper(x):  
        if x <= 0:  
            raise ValueError(f"Аргумент должен быть > 0, получено: {x}")  
        return func(x)  
    return wrapper  
  
@require_positive  
def square_root(x):  
    return x ** 0.5  
  
print(square_root(9))      # 3.0  
print(square_root(-4))    # ValueError!
```

 **Задание 2:** Напиши декоратор `@log_call`, который перед вызовом любой функции печатает: «Вызов функции [имя] с аргументами [args]». Применни к 2-3 функциям.

Глава 17: Алгоритмы сортировки и поиска

17.1 Сложность алгоритмов (Big O)

Big O показывает, как растёт время работы алгоритма при увеличении данных.

Нотация	Название	Пример при N=1000
$O(1)$	Константная	~1 операция
$O(\log N)$	Логарифмическая	~10 операций
$O(N)$	Линейная	~1 000 операций
$O(N \log N)$	Линейно-логарифмическая	~10 000 операций
$O(N^2)$	Квадратичная	~1 000 000 операций

17.2 Сортировка пузырьком

Простейший алгоритм — сравниваем соседние элементы и меняем местами. Сложность $O(N^2)$.

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr

nums = [64, 34, 25, 12, 22, 11, 90]
print(bubble_sort(nums))  # [11, 12, 22, 25, 34, 64, 90]
```

17.3 Быстрая сортировка (QuickSort)

Разделяй и властвуй — выбираем опорный элемент, делим массив. Средняя сложность $O(N \log N)$.

```
def quicksort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]  # опорный элемент
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quicksort(left) + middle + quicksort(right)

nums = [3, 6, 8, 10, 1, 2, 1]
print(quicksort(nums))  # [1, 1, 2, 3, 6, 8, 10]
```

17.4 Двоичный поиск

Ищем в отсортированном массиве — каждый раз отсекаем половину. Сложность $O(\log N)$.

```
def binary_search(arr, target):
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid          # нашли!
        elif arr[mid] < target:
            left = mid + 1      # ищем правее
        else:
            right = mid - 1     # ищем левее
    return -1                  # не найдено

data = [1, 3, 5, 7, 9, 11, 13, 15]
print(binary_search(data, 7))    # 3   (индекс)
print(binary_search(data, 6))    # -1  (не найдено)
```

```
3
-1
```

□ **Python и сортировка:** В реальном коде используй встроенную `sorted()` или `list.sort()` — они реализованы на C и намного быстрее. Алгоритмы выше — для понимания принципов.

 **Задание 3:** Реализуй рекурсивный бинарный поиск. Напиши функцию `binary_search_recursive(arr, target, left, right)`. Сравни результаты с итерационной версией.

Глава 18: Работа с CSV и JSON

18.1 Модуль csv

CSV (Comma-Separated Values) — табличные данные в текстовом файле. Открывается в Excel. Python умеет читать и писать CSV.

```
import csv

# Запись CSV
students = [
    ["Имя", "Класс", "Оценка"],
    ["Алёша", "9А", 95],
    ["Маша", "9Б", 88],
    ["Дима", "9А", 72],
]

with open("students.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerows(students)

# Чтение CSV
with open("students.csv", "r", encoding="utf-8") as f:
    reader = csv.DictReader(f)  # каждая строка — словарь
    for row in reader:
        print(f"{row['Имя']:10} — {row['Оценка']} баллов")
```

Алёша	— 95 баллов
Маша	— 88 баллов
Дима	— 72 баллов

18.2 Модуль json

JSON — универсальный формат обмена данными. Используется в API, конфигурационных файлах, базах данных.

```
import json

data = {
    "app": "MyApp",
    "version": "2.1",
    "users": [
        {"id": 1, "name": "Анна", "active": True},
        {"id": 2, "name": "Борис", "active": False},
    ]
}

# Запись в файл
with open("config.json", "w", encoding="utf-8") as f:
    json.dump(data, f, ensure_ascii=False, indent=2)

# Чтение из файла
with open("config.json", "r", encoding="utf-8") as f:
    loaded = json.load(f)

print(loaded["app"])          # MyApp
print(loaded["users"][0]["name"]) # Анна

# Строка ↔ JSON
text = json.dumps(data, ensure_ascii=False)
obj = json.loads(text)
```

Функция	Описание
<code>json.dump(obj, file)</code>	Записать объект в файл
<code>json.load(file)</code>	Прочитать объект из файла
<code>json.dumps(obj)</code>	Объект → строка JSON
<code>json.loads(text)</code>	Строка JSON → объект

 **Задание 4:** Создай JSON-файл с каталогом книг (минимум 5 книг): название, автор, год, жанр. Напиши функцию поиска книг по жанру, выводи результаты красиво.

Глава 19: Запросы к интернету (requests)

19.1 Установка и основы

Библиотека `requests` — самый популярный способ делать HTTP-запросы в Python.

```
# В терминале:
pip install requests

import requests

# GET-запрос
response = requests.get("https://api.github.com")

print(response.status_code)    # 200 (успех)
print(response.headers["Content-Type"])
data = response.json()         # разбираем JSON-ответ
print(data["current_user_url"])
```

Код	Значение
200	OK — запрос выполнен
404	Not Found — страница не найдена
403	Forbidden — нет доступа
500	Internal Server Error — ошибка сервера

19.2 Работа с открытыми API

Пример: получаем данные о погоде через бесплатный API `wtr.in`

```
import requests
import json

def get_weather(city):
    url = f"https://wttr.in/{city}?format=json"
    try:
        response = requests.get(url, timeout=5)
        response.raise_for_status() # исключение если ошибка
        data = response.json()
        current = data["current_condition"][0]
        temp_c = current["temp_C"]
        feels = current["FeelsLikeC"]
        desc = current["weatherDesc"][0]["value"]
        print(f"Погода в {city}:")
        print(f"  Температура: {temp_c}°C (ощущается как {feels}°C)")
        print(f"  Состояние: {desc}")
    except requests.exceptions.ConnectionError:
        print("Нет соединения с интернетом")
    except requests.exceptions.Timeout:
        print("Сервер не отвечает (timeout)")

get_weather("Moscow")
get_weather("London")
```

19.3 POST-запрос с данными

```
import requests

# Пример: отправка данных на тестовый сервер
url = "https://httpbin.org/post"
payload = {"username": "student", "score": 100}

response = requests.post(url, json=payload)
result = response.json()
print(result["json"]) # {"username": "student", "score": 100}
```

 **API-ключи:** Многие API требуют ключ доступа. Никогда не вставляй ключ прямо в код! Храни в переменных окружения или файле `.env`, который не заливается в git.

 **Задание 5:** Напиши программу, которая спрашивает название города и выводит текущую погоду. Добавь обработку ошибок — на случай если город не найден.

Глава 20: Базы данных SQLite

20.1 Что такое SQLite?

SQLite — лёгкая база данных, встроенная прямо в Python. Не нужно ничего устанавливать. Данные хранятся в одном .db файле.

```
import sqlite3

# Создаём или открываем базу данных
conn = sqlite3.connect("school.db")
cursor = conn.cursor()

# Создаём таблицу
cursor.execute("""
    CREATE TABLE IF NOT EXISTS students (
        id        INTEGER PRIMARY KEY AUTOINCREMENT,
        name       TEXT     NOT NULL,
        grade      TEXT,
        score      INTEGER DEFAULT 0
    )
""")

conn.commit()    # сохраняем изменения
print("База данных создана!")
conn.close()
```

20.2 CRUD-операции

CRUD = Create, Read, Update, Delete — четыре основных операции с данными.

```
import sqlite3

conn = sqlite3.connect("school.db")
cursor = conn.cursor()

# CREATE — добавить записи
cursor.execute("INSERT INTO students (name, grade, score) VALUES (?, ?, ?)",
               ("Алёша", "9A", 95))
cursor.executemany("INSERT INTO students (name, grade, score) VALUES (?, ?, ?)", [
    ("Маша", "9Б", 88),
    ("Дима", "9A", 72),
    ("Катя", "9Б", 91),
])

# READ — прочитать данные
cursor.execute("SELECT * FROM students WHERE score >= 90")
rows = cursor.fetchall()
for row in rows:
    print(row)    # (id, name, grade, score)

# UPDATE — обновить
cursor.execute("UPDATE students SET score = 98 WHERE name = ?", ("Алёша",))

# DELETE — удалить
cursor.execute("DELETE FROM students WHERE score < 75")

conn.commit()
conn.close()
```

20.3 Класс-обёртка для БД

```
import sqlite3
from contextlib import contextmanager

class StudentDB:
    def __init__(self, path="students.db"):
        self.path = path
        self._init_db()

    @contextmanager
    def _connect(self):
        conn = sqlite3.connect(self.path)
        conn.row_factory = sqlite3.Row # строки как словари
        try:
            yield conn
            conn.commit()
        finally:
            conn.close()

    def _init_db(self):
        with self._connect() as conn:
            conn.execute("""CREATE TABLE IF NOT EXISTS students
                (id INTEGER PRIMARY KEY AUTOINCREMENT,
                 name TEXT, grade TEXT, score INTEGER)""")

    def add(self, name, grade, score):
        with self._connect() as conn:
            conn.execute("INSERT INTO students (name,grade,score)
VALUES(?,?,?)",
                        (name, grade, score))

    def get_all(self):
        with self._connect() as conn:
            return conn.execute("SELECT * FROM students ORDER BY score
DESC").fetchall()

    def top_students(self, limit=3):
        with self._connect() as conn:
            return conn.execute(
                "SELECT name, score FROM students ORDER BY score DESC LIMIT
?",
                (limit,)
            ).fetchall()

# Использование:
db = StudentDB()
db.add("Анна", "10А", 97)
db.add("Илья", "10В", 84)
db.add("Света", "10А", 91)

print("Топ-3 учеников:")
for row in db.top_students():
    print(f" {row['name']:10} - {row['score']} баллов")
```

 **Задание 6:** Создай БД для библиотеки: таблица books (id, title, author, year, available). Напиши функции: добавить книгу, взять книгу (available=False), вернуть книгу, показать доступные.

Глава 21: Многопоточность

21.1 Зачем нужна многопоточность?

Когда программа ждёт ответа от интернета или читает файл, процессор простаивает. Потоки позволяют делать несколько дел «одновременно».

```
import threading
import time

def download_file(name, delay):
    print(f"Начинаю загрузку: {name}")
    time.sleep(delay)    # имитируем загрузку
    print(f"Загружено:      {name}")

# Без потоков — последовательно (займёт 6 сек)
# download_file("file1.zip", 2)
# download_file("file2.zip", 3)
# download_file("file3.zip", 1)

# С потоками — параллельно (займёт ~3 сек)
threads = [
    threading.Thread(target=download_file, args=("file1.zip", 2)),
    threading.Thread(target=download_file, args=("file2.zip", 3)),
    threading.Thread(target=download_file, args=("file3.zip", 1)),
]

start = time.time()
for t in threads:
    t.start()
for t in threads:
    t.join()    # ждём завершения всех потоков

print(f"Всего: {time.time()-start:.1f} сек")
```

```
Начинаю загрузку: file1.zip
Начинаю загрузку: file2.zip
Начинаю загрузку: file3.zip
Загружено:      file3.zip
Загружено:      file1.zip
Загружено:      file2.zip
Всего: 3.0 сек
```

21.2 ThreadPoolExecutor (современный подход)

```
from concurrent.futures import ThreadPoolExecutor
import requests

urls = [
    "https://httpbin.org/delay/1",
    "https://httpbin.org/delay/2",
    "https://httpbin.org/delay/1",
]

def fetch(url):
    r = requests.get(url, timeout=10)
    return f"{url} → {r.status_code}"

# Запускаем 3 запроса параллельно
with ThreadPoolExecutor(max_workers=3) as executor:
    results = list(executor.map(fetch, urls))

for r in results:
    print(r)
```

⚠ GIL в Python: Python имеет Global Interpreter Lock — потоки не ускоряют CPU-задачи (вычисления). Они полезны для I/O (сеть, файлы). Для вычислений используй multiprocessing.

Глава 22: Тестирование кода (unittest)

22.1 Зачем тестировать?

Тесты — это код, который проверяет твой код. Профессиональные разработчики пишут тесты обязательно. Тесты помогают найти ошибки и убедиться, что изменения ничего не сломали.

22.2 Модуль unittest

```
import unittest

# Функции, которые тестируем
def add(a, b):
    return a + b
def divide(a, b):
    if b == 0: raise ValueError("На ноль делить нельзя")
    return a / b
def is_palindrome(s):
    return s == s[::-1]

class TestMathFunctions(unittest.TestCase):

    def test_add_positive(self):
        self.assertEqual(add(2, 3), 5)

    def test_add_negative(self):
        self.assertEqual(add(-1, -1), -2)

    def test_add_float(self):
        self.assertAlmostEqual(add(0.1, 0.2), 0.3, places=10)

    def test_divide_normal(self):
        self.assertEqual(divide(10, 2), 5.0)

    def test_divide_by_zero(self):
        with self.assertRaises(ValueError):
            divide(5, 0)          # ожидаем ValueError

    def test_palindrome_true(self):
        self.assertTrue(is_palindrome("рояль"))
        # рояль — не палиндром, тест УПАДЁТ — это намеренно!

    def test_palindrome_false(self):
        self.assertFalse(is_palindrome("python"))

if __name__ == "__main__":
    unittest.main(verbosity=2)
```



```

test_add_float ... ok
test_add_negative ... ok
test_add_positive ... ok
test_divide_by_zero ... ok
test_divide_normal ... ok
test_palindrome_false ... ok
test_palindrome_true ... FAIL ← намеренная ошибка
-----
Ran 7 tests in 0.002s
FAILED (failures=1)

```

Метод	Проверяет что...
<code>assertEqual(a, b)</code>	<code>a == b</code>
<code>assertNotEqual(a, b)</code>	<code>a != b</code>
<code>assertTrue(x)</code>	x истинно
<code>assertFalse(x)</code>	x ложно
<code>assertRaises(Exc, fn)</code>	fn() вызывает исключение Exc
<code>assertIn(a, b)</code>	a содержится в b
<code>assertAlmostEqual(a, b)</code>	$a \approx b$ (с точностью)

 **Задание 7:** Напиши класс Calculator с методами add, subtract, multiply, divide. Создай TestCalculator с минимум 10 тестами, включая граничные случаи и проверку исключений.

□ Итоговый проект: Менеджер задач с БД

Консольный менеджер задач (To-Do) с хранением в SQLite, приоритетами, дедлайнами и статистикой.

```
import sqlite3
from contextlib import contextmanager

class TaskManager:
    def __init__(self, db_path='tasks.db'):
        self.db = db_path
        self._migrate()

    @contextmanager
    def _conn(self):
        c = sqlite3.connect(self.db)
        c.row_factory = sqlite3.Row
        try:
            yield c; c.commit()
        finally:
            c.close()

    def _migrate(self):
        with self._conn() as c:
            c.execute("CREATE TABLE IF NOT EXISTS tasks("+
                        "id INTEGER PRIMARY KEY AUTOINCREMENT, "+
                        "title TEXT NOT NULL, priority TEXT DEFAULT
'medium', "+
                        "due_date TEXT, done INTEGER DEFAULT 0)")

    def add(self, title, priority='medium', due=None):
        with self._conn() as c:
            c.execute('INSERT INTO tasks(title,priority,due_date)
VALUES(?,?,?)',
                    (title, priority, due))
            print(f'Добавлено: {title}')

    def complete(self, task_id):
        with self._conn() as c:
            c.execute('UPDATE tasks SET done=1 WHERE id=?', (task_id,))

    def list_tasks(self, show_done=False):
        with self._conn() as c:
            q = 'SELECT * FROM tasks'
            if not show_done: q += ' WHERE done=0'
            return c.execute(q).fetchall()

    def stats(self):
        with self._conn() as c:
            total = c.execute('SELECT COUNT(*) FROM tasks').fetchone()[0]
            done = c.execute('SELECT COUNT(*) FROM tasks WHERE
done=1').fetchone()[0]
            return {'total': total, 'done': done, 'active': total-done}

    def show_tasks(tasks):
        if not tasks: print('Нет задач!'); return
        for t in tasks:
            s = 'ТОДО' if t['done'] else '-----'
            print(t['id'], s, t['title'], t['due_date'] or '-')

def main():
```

```
tm = TaskManager()
while True:
    print('1-Список 2-Добавить 3-Выполнить 0-Выход')
    c = input('Выбор: ').strip()
    if c == '1': show_tasks(tm.list_tasks())
    elif c == '2':
        t = input('Название: ')
        p = input('Приоритет (low/medium/high): ') or 'medium'
        d = input('Дедлайн (ГГГГ-ММ-ДД или Enter): ') or None
        tm.add(t, p, d)
    elif c == '3':
        show_tasks(tm.list_tasks())
        tm.complete(int(input('Номер: ')))
    elif c == '0': print('До встречи!'); break

main()
```

□ **Что использовано в проекте:** SQLite (хранение) • Классы и ООП • Декоратор contextmanager • f-строки • datetime • Обработка ошибок • Форматированный вывод

 **Задание 8:** Расширь проект: добавь экспорт задач в JSON-файл и импорт из него. Добавь поиск задач по ключевому слову. Добавь категории (работа/учёба/личное).

□ Карта развития Python-разработчика

Выбери направление:

Направление	Что изучать	Библиотеки
□ Веб-разработка	HTTP, HTML, REST API	Django, Flask, FastAPI
□ Анализ данных	Статистика, визуализация	Pandas, NumPy, Matplotlib
□ ML / AI	Машинное обучение	scikit-learn, TensorFlow, PyTorch
□ Разработка игр	Физика, графика, звук	Pygame, Arcade
□ Кибербезопасность	Сети, криптография	Scapy, PyCryptodome
□ Автоматизация	Парсинг, боты, скрипты	Selenium, BeautifulSoup, aiogram
□ Десктоп-приложения	GUI, события, виджеты	Tkinter, PyQt6, customtkinter

Полезные ресурсы

- docs.python.org — официальная документация
- stepik.org — русскоязычные курсы по Python
- leetcode.com / codeforces.com — задачи на алгоритмы
- realpython.com — статьи и tutorиалы
- github.com — смотри исходный код реальных проектов
- stackoverflow.com — ответы на любые вопросы